الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الثالث عشر والأخير

فصل المقارنات الشاملة وWorkflow احترافي متكامل

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة الجزء الختامي 3
الفصل الأول: مقارنة شاملة بين المكتبات الست 4
١.١ الخريطة العامة: من يفعل ماذا 4
١.٢ سلسلة الاعتماديات (Dependency Chain) 4
١.٣ متى تختار كل مكتبة: جدول قرار موسّع 4
١.٤ الرسم البياني: Matplotlib مقابل Plotly مقابل Seaborn 5
١.٥ أخطاء شائعة في اختيار الأداة الخطأ 5
ملخص الفصل 5
الفصل الثاني: مشروع Workflow احترافي متكامل 6
٢.١ هيكلة المشروع الكاملة 6
٢.٢ الخطوة ١: تحميل وتنظيف البيانات (Pandas + NumPy) 6
٢.٣ الخطوة ٢: استكشاف بصري (Matplotlib + Plotly داخل Jupyter) 7
٢.٤ الخطوة ٣: بناء النموذج (Scikit-learn) 7
٢.٥ الخطوة ٤: التقييم والتصوير (Scikit-learn + Matplotlib) 8
٢.٦ الخطوة ٥: التقرير النهائي (Jupyter + nbconvert) 8
٢.٧ ملخص تدفق البيانات عبر المكتبات الست 8
أفضل الممارسات 9
تمارين عملية 9
حلول مقترحة (تلميحات) 9
خاتمة الكتاب 9

مقدمة الجزء الختامي

بعد اثني عشر جزءًا غطّت كل مكتبة على حدة بعمق، يجمع هذا الجزء الأخير كل الخيوط معًا: مقارنة مباشرة بين المكتبات الست حول متى تختار كل واحدة، ثم مشروع Workflow احترافي واحد متكامل يُظهر كيف تتعاون NumPy وPandas وMatplotlib وPlotly وScikit-learn وJupyter معًا في خط أنابيب علم بيانات حقيقي واحد من البيانات الخام حتى النموذج النهائي والتقرير المُصدَّر.

الفصل الأول: مقارنة شاملة بين المكتبات الست

١.١ الخريطة العامة: من يفعل ماذا

المكتبة	الدور الأساسي	المدخل	المخرج
NumPy	الحساب العددي الخام على مصفوفات متجانسة	أرقام / مصفوفات	مصفوفات NumPy
Pandas	تنظيم وتنظيف وتحليل البيانات الجدولية	مصفوفات/CSV/Excel/SQL NumPy	DataFrame/Series
Matplotlib	رسم بياني ثابت عالي الدقة	مصفوفات NumPy / أعمدة Pandas	صور PNG/SVG/PDF
Plotly	رسم بياني تفاعلي للويب ولوحات المعلومات	مصفوفات / DataFrame	HTML تفاعلي / صور
Scikit-learn	بناء وتقييم نماذج تعلّم آلي تقليدي	مصفوفات/DataFrame عددية	نموذج مدرَّب + تنبؤات
Jupyter	بيئة تشغيل تفاعلية تجمع كل ما سبق	كود Python + Markdown	دفتر تفاعلي / تقرير مُصدَّر

١.٢ سلسلة الاعتماديات (Dependency Chain)

من المهم إدراك أن هذه المكتبات ليست ستة خيارات منفصلة متكافئة، بل هرم اعتماديات مبني بعضه فوق بعض: Pandas تعتمد على NumPy مباشرة، Matplotlib تتكامل مع كليهما، Scikit-learn تتوقع مدخلات على هيئة مصفوفات NumPy (أو DataFrame يُحوَّل ضمنيًا)، وPlotly تتكامل غالبًا مع DataFrame من Pandas مباشرة. أما Jupyter فهو الحاوية التي تُشغَّل بداخلها كل هذه المكتبات تفاعليًا.

```
Jupyter (بيئة التشغيل)
  └── NumPy (الأساس العددي)
        └── Pandas (بنية جدولية فوق NumPy)
              ├── Matplotlib (رسم ثابت، يقرأ من NumPy/Pandas)
              ├── Plotly (رسم تفاعلي، يقرأ من Pandas غالبًا)
              └── Scikit-learn (نمذجة، تتوقع مصفوفات NumPy/DataFrame)
```

١.٣ متى تختار كل مكتبة: جدول قرار موسّع

المكتبة الأنسب	السيناريو
NumPy	عملية حسابية بحتة على مصفوفات أرقام متجانسة
Pandas	قراءة CSV وتنظيف بيانات بأعمدة مختلطة الأنواع
Matplotlib	رسم جاهز للنشر الأكاديمي أو تقرير PDF مطبوع
Plotly + Dash	لوحة معلومات ويب تفاعلية حية لفريق أو عملاء
Scikit-learn	تصنيف أو انحدار أو تجميع على بيانات جدولية
Jupyter	استكشاف بيانات جديدة تفاعليًا خطوة بخطوة
Dask أو Polars (خارج نطاق هذا الكتاب)	بيانات أكبر من الذاكرة المتاحة

المكتبة الأنسب	السيناريو
PyTorch/TensorFlow (خارج نطاق هذا الكتاب)	شبكات عصبية عميقة أو صور/نصوص معقدة

١.٤ الرسم البياني: Matplotlib مقابل Plotly مقابل Seaborn

Seaborn (إضافية)	Plotly	Matplotlib	المعيار
لا (فوق Matplotlib)	ممتازة افتراضيًا	ضعيفة افتراضيًا	التفاعلية
الأسرع (سطر واحد)	سريعة عبر express	متوسطة	سرعة الرسم الإحصائي القياسي
جيدة جدًا	جيدة لكن أقل شيوعًا	المعيار الذهبي	جودة النشر الأكاديمي
غير مباشر	الخيار الطبيعي	غير مباشر	الاستخدام في تطبيقات ويب
الأسهل للرسوم الإحصائية	سهل عبر express	متوسط إلى مرتفع للتخصيص الدقيق	منحنى التعلّم

قاعدة عملية مختصرة: Seaborn للاستكشاف الإحصائي الفوري بأقل كود، Matplotlib للنشر الثابت عالي الدقة، Plotly عندما تكون التفاعلية أو الويب هدفًا أساسيًا.

١.٥ أخطاء شائعة في اختيار الأداة الخطأ

- استخدام Pandas لعمليات حسابية بحتة على مصفوفات متجانسة كبيرة بدل NumPy مباشرة، مما يضيف Overhead غير ضروري.
- محاولة بناء لوحة معلومات ويب تفاعلية كاملة فوق Matplotlib بدل التحول مباشرة لـ Plotly/Dash المصممتين لهذا الغرض أصلاً.
- استخدام Scikit-learn لمهام تتطلب شبكات عصبية عميقة (تصنيف صور معقد، معالجة لغة طبيعية متقدمة) بدل أطر التعلّم العميق المتخصصة.
- كتابة كود إنتاجي كامل داخل دفاتر Jupyter دون استخراج المنطق المستقر لوحدات .py منظمة وقابلة للاختبار.

ملخص الفصل

- المكتبات الست تشكّل هرم اعتماديات وليست خيارات منفصلة: NumPy في القاعدة، Jupyter كحاوية تشغيل للجميع.
- اختيار الأداة الصحيحة يعتمد على طبيعة المهمة (حساب بحت، بيانات جدولية، رسم ثابت أو تفاعلي، نمذجة، استكشاف) لا على التفضيل الشخصي فقط.
- أشهر خطأ هو استخدام أداة مصممة لغرض مختلف تمامًا عن الحاجة الفعلية، مما يضيف تعقيدًا أو يفوّت أداءً أفضل متاحًا في أداة أخرى.

الفصل الثاني: مشروع Workflow احترافي متكامل

هذا المشروع الختامي يطبّق المكتبات الست معًا في خط أنابيب واحد متكامل، من بيانات خام وهمية حتى نموذج مدرَّب وتقرير HTML نهائي — كل ذلك داخل بنية مشروع منظّمة كما نوصي بها طوال هذا الكتاب. الهدف ليس تعليم خوارزمية جديدة، بل إظهار كيف تتحاور المكتبات الست فعليًا في مشروع واحد حقيقي.

٢.١ هيكلة المشروع الكاملة

```
customer_churn_project/
├── data/
│   ├── raw/customers.csv
│   └── processed/customers_clean.parquet
├── src/
│   ├── __init__.py
│   ├── load.py            # قراءة البيانات الخام (Pandas)
│   ├── clean.py           # تنظيف وتحويل (Pandas + NumPy)
│   ├── features.py        # ColumnTransformer (Scikit-learn)
│   ├── model.py            # Pipeline كامل +تدريب (Scikit-learn)
│   └── viz.py               # دوال رسم قابلة لإعادة الاستخدام (Matplotlib/Plotly)
├── notebooks/
│   ├── 01_exploration.ipynb
│   ├── 02_modeling.ipynb
│   └── 03_final_report.ipynb
├── models/model_v1.joblib
├── tests/
└── requirements.txt
```

٢.٢ الخطوة ١: تحميل وتنظيف البيانات (Pandas + NumPy)

```
# src/load.py
import pandas as pd

def load_raw_data(path: str) -> pd.DataFrame:
    return pd.read_csv(path, parse_dates=['signup_date'])

# src/clean.py
import numpy as np

def clean_customers(df: pd.DataFrame) -> pd.DataFrame:
    return (
        df
        .drop_duplicates(subset='customer_id')
        .assign(
            tenure_days=lambda d: (pd.Timestamp.now() - d['signup_date']).dt.days,
            monthly_charges=lambda d: d['monthly_charges'].clip(lower=0),
        )
        .dropna(subset=['customer_id'])
        .reset_index(drop=True)
    )
```

هنا يظهر التكامل الأول: Pandas تتولى القراءة والتنظيف عبر Method Chaining، بينما NumPy تعمل بصمت خلف الكواليس داخل كل عملية حسابية على الأعمدة (فكل عمود Series هو أصلاً مصفوفة NumPy من الداخل).

٢.٣ الخطوة ٢: استكشاف بصري (Matplotlib + Plotly داخل Jupyter)

```
# notebooks/01_exploration.ipynb
from src.load import load_raw_data
from src.clean import clean_customers
import matplotlib.pyplot as plt
import plotly.express as px

df = clean_customers(load_raw_data('data/raw/customers.csv'))

# استكشاف سريع ثابت (Matplotlib) لفحص أولي داخلي
fig, ax = plt.subplots()
df['monthly_charges'].hist(bins=30, ax=ax)
ax.set_title('توزيع الفاتورة الشهرية')

# رسم تفاعلي (Plotly) لاستكشاف أعمق قابل للتصفية بصريًا
px.box(df, x='contract_type', y='tenure_days', color='churned')
```

النمط الاحترافي هنا: Matplotlib للفحص السريع الداخلي أثناء العمل الفردي، وPlotly عند الحاجة لاستكشاف تفاعلي أعمق أو لمشاركة النتيجة مع الفريق لاحقًا كملف HTML قابل للتمرير والتصفية.

٢.٤ الخطوة ٣: بناء النموذج (Scikit-learn)

```
# src/features.py
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

def build_preprocessor():
    return ColumnTransformer([
        ('num', StandardScaler(), ['tenure_days', 'monthly_charges']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['contract_type']),
    ])

# src/model.py
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import joblib
from src.features import build_preprocessor

def train_model(df):
    X = df[['tenure_days', 'monthly_charges', 'contract_type']]
    y = df['churned']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    pipe = Pipeline([
        ('preprocess', build_preprocessor()),
```

```
        ('model', RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-
1)),
    ])
    pipe.fit(X_train, y_train)
    joblib.dump(pipe, 'models/model_v1.joblib')
    return pipe, X_test, y_test
```

لاحظ أن كل المدخلات هنا هي عمليًا DataFrame من Pandas؛ Scikit-learn تتعامل معها بسلاسة تامة بفضل التكامل الطبيعي بين المكتبتين، محوّلة إياها داخليًا إلى مصفوفات NumPy عند الحاجة الفعلية للحساب.

٢.٥ الخطوة ٤: التقييم والتصوير (Scikit-learn + Matplotlib)

```
# src/viz.py
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

def plot_confusion(pipe, X_test, y_test):
    fig, ax = plt.subplots(figsize=(5, 5))
    ConfusionMatrixDisplay.from_estimator(pipe, X_test, y_test, ax=ax)
    ax.set_title('مصفوفة الالتباس - توقّع مغادرة العملاء')
    return fig
```

ConfusionMatrixDisplay مثال ممتاز على التكامل العميق بين Scikit-learn وMatplotlib: أداة تقييم من sklearn.metrics تُرجع كائن Matplotlib Figure/Axes جاهزًا مباشرة، دون الحاجة لبناء الرسم يدويًا من الصفر.

٢.٦ الخطوة ٥: التقرير النهائي (Jupyter + nbconvert)

```
# notebooks/03_final_report.ipynb
from src.load import load_raw_data
from src.clean import clean_customers
from src.model import train_model
from src.viz import plot_confusion

df = clean_customers(load_raw_data('data/raw/customers.csv'))
pipe, X_test, y_test = train_model(df)
plot_confusion(pipe, X_test, y_test)

# بعد، من الطرفية Restart & Run All ناجح:
jupyter nbconvert --to html --no-input notebooks/03_final_report.ipynb --output
../final_report.html
```

الدفتر النهائي لا يحتوي أي منطق معقّد مباشر — فقط استدعاءات لدوال مستقرة من src/، تمامًا كما أوصينا في فصل أفضل ممارسات Jupyter. هذا يضمن أن التقرير قابل لإعادة التوليد بالكامل بضغطة واحدة (Restart & Run All ثم nbconvert)، وهو تعريف 'قابلية إعادة الإنتاج' الذي شرحناه سابقًا مطبّقًا عمليًا بالكامل.

٢.٧ ملخص تدفق البيانات عبر المكتبات الست

الخطوة	المكتبة الرئيسية	المدخل	المخرج
١. تحميل وتنظيف	Pandas (+ NumPy ضمنيًا)	خام CSV	نظيف DataFrame
٢. استكشاف بصري	Matplotlib + Plotly (داخل Jupyter)	نظيف DataFrame	رسوم استكشافية
٣. بناء النموذج	Scikit-learn	نظيف DataFrame	Pipeline مدرَّب (.joblib)
٤. التقييم والتصوير	Scikit-learn + Matplotlib	بيانات اختبار + Pipeline	مقاييس ورسوم تقييم
٥. التقرير النهائي	Jupyter + nbconvert	كل ما سبق مجمَّعًا	تقرير HTML نهائي

أفضل الممارسات

- افصل دائمًا منطق كل مرحلة (تحميل، تنظيف، ميزات، نمذجة، تصوير) في وحدة .py مستقلة قابلة للاختبار وإعادة الاستخدام.
- استخدم الدفاتر فقط للتنسيق والاستدعاء والعرض، لا لاحتضان منطق معقّد مباشر بمجرد استقراره.
- وثّق تدفق البيانات بين المراحل (كما في الجدول أعلاه) في README المشروع لأي عضو فريق جديد.
- اختتم أي مشروع بـ Restart & Run All على الدفتر النهائي قبل تصديره، كفحص سلامة أخير قبل التسليم.

تمارين عملية

١. طبّق هذا الـ Workflow الكامل على بيانات من اختيارك: نظّف عبر Pandas، استكشف عبر Matplotlib/Plotly، درّب نموذجًا عبر Scikit-learn، وصدّر تقريرًا عبر Jupyter/nbconvert.

٢. أضف مرحلة سادسة لهذا الـ Workflow: حفظ ملخص أداء النموذج (accuracy, f1) في ملف JSON منفصل قابل للتتبع عبر تجارب متعددة.

٣. حوّل رسم مصفوفة الالتباس الثابت (Matplotlib) في هذا المشروع إلى رسم تفاعلي مكافئ عبر Plotly.

حلول مقترحة (تلميحات)

١. اتبع بنية src/ المذكورة أعلاه خطوة بخطوة مع بياناتك الخاصة، مع إبقاء كل دالة في ملفها المناسب.

٢. أضف دالة log_experiment(params, metrics) في src/model.py تكتب سجلاً بصيغة JSON lines كما في فصل Jupyter السابع.

٣. استخدم px.imshow على مصفوفة confusion_matrix(y_test, y_pred) مباشرة بدل ConfusionMatrixDisplay.

خاتمة الكتاب

بهذا يكتمل هذا المرجع الاحترافي الشامل عبر ١٣ جزءًا: ست مكتبات أساسية في علم البيانات والتعلّم الآلي بلغة بايثون، كل واحدة مغطّاة بعمق عبر أحد عشر فصلاً من التاريخ والفلسفة حتى المشاريع المصغّرة، وختامًا بمقارنة شاملة وWorkflow متكامل يربط كل شيء معًا.

الخطوة التالية الطبيعية بعد هذا الكتاب هي التطبيق العملي المستمر: خذ بيانات حقيقية من اهتمامك الشخصي أو عملك، وطبّق نفس الـ Workflow المذكور في هذا الفصل الختامي من الألف إلى الياء. الإتقان الحقيقي لهذه المكتبات لا يأتي من القراءة وحدها، بل من مواجهة البيانات الفوضوية الحقيقية وحل مشاكلها خطوة بخطوة بنفس الأدوات التي أتقنتها للتو.